

MI VIF API

Version 3.5

©2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020
3.1	Added Muffin flow char	07/16/2021
3.2	Added procfs introduction	08/25/2021
3.3	- Update MI_VIF_GroupIdMask_e E_MI_VIF_GROUPMASK_ID up to 7 - Update char flow note info	12/22/2021
3.4	Add Mochi information	02/17/2022
3.5	Add Maruko information	03/21/2022

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. 概述	1
1.1. 模块说明	1
1.2. 流程框图	1
1.2.1 Tiramisu 框图	1
1.2.2 Muffin 框图	2
1.2.3 Mochi 框图	4
1.2.4 Maruko 框图	5
1.3. 关键字说明	6
2. API 参考	7
2.1. MI_VIF_CreateDevGroup	7
2.2. MI_VIF_CreateDevGroupExt	11
2.3. MI_VIF_DestroyDevGroup	12
2.4. MI_VIF_GetDevGroupAttr	13
2.5. MI_VIF_SetDevAttr	13
2.6. MI_VIF_GetDevAttr	14
2.7. MI_VIF_EnableDev	15
2.8. MI_VIF_DisableDev	16
2.9. MI_VIF_GetDevStatus	17
2.10. MI_VIF_SetOutputPortAttr	17
2.11. MI_VIF_GetOutputPortAttr	19
2.12. MI_VIF_EnableOutputPort	19
2.13. MI_VIF_DisableOutputPort	20
2.14. MI_VIF_Query	21
2.15. MI_VIF_CallBackTask_Register	22
2.16. MI_VIF_CallBackTask_UnRegister	24
3. VIF 数据类型	25
3.1. MI_VIF_GROUP	25
3.2. MI_VIF_IntfMode_e	26
3.3. MI_VIF_WorkMode_e	27
3.4. MI_VIF_FrameRate_e	27
3.5. MI_VIF_ClkEdge_e	28
3.6. MI_VIF_HDRTYPE_e	28
3.7. MI_VIF_MclkSource_e	29
3.8. MI_VIF_GroupIdMask_e	30
3.9. MI_VIF_SNRPad_e	31
3.10. MI_VIF_GroupAttr_t	31
3.11. MI_VIF_GroupExtAttr_t	32
3.12. MI_VIF_DevAttr_t	33
3.13. MI_VIF_OutputPortAttr_t	34
3.14. MI_VIF_DevPortStat_t	34
3.15. MI_VIF_DevStatus_t	35

3.16. MI_VIF_CALLBK_FUNC.....	36
3.17. MI_VIF_CallBackMode_e.....	36
3.18. MI_VIF_IrqType_e.....	37
3.19. MI_VIF_CallBackParam_t.....	37
4. VIF 错误码.....	39
5. PROCFS 介绍.....	40
5.1. cat.....	40
5.2. echo.....	41

1. 概述

1.1. 模块说明

视频输入（VIF）实现启用视频输入设备、视频输入通道、绑定视频输入通道等功能。

1.2. 流程框图

1.2.1 Tiramisu 框图

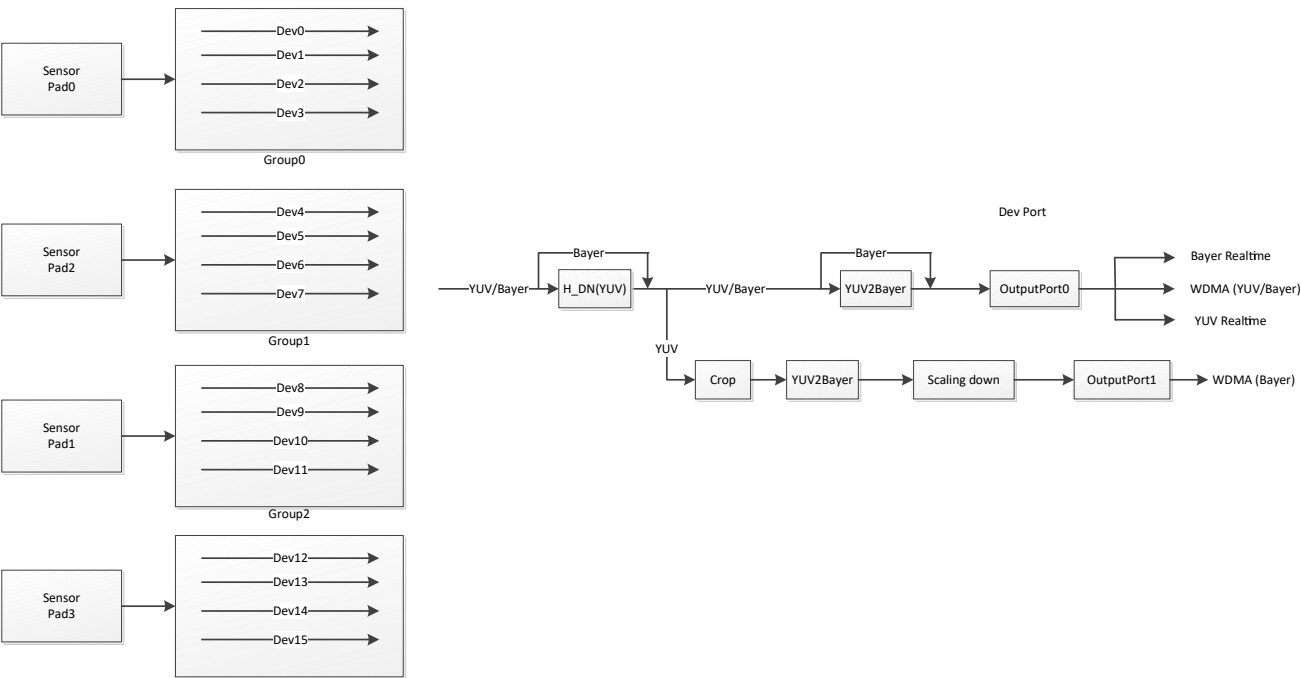


图 1-1: Tiramisu 框图

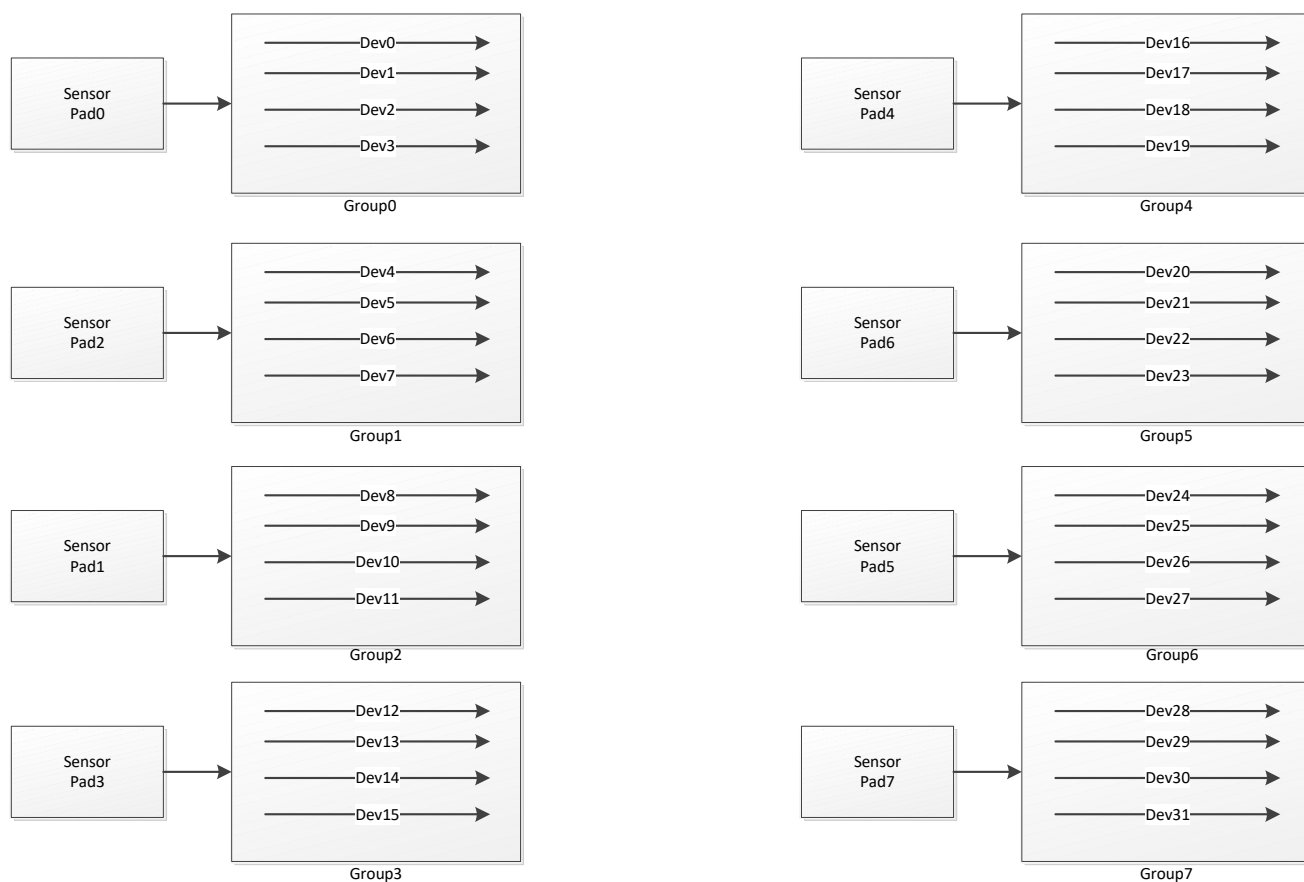
※ 注意

- 16 个 Dev 中总共只有 8 个 Dev 可以用。
- mipi 接口 4 个 sensor 分别用到：
linear mode: dev0/dev4/dev8/dev12,
hdr frame mode: dev0/1, dev4/5, dev8/9, dev12/13。
hdr realtime mode: dev0/dev1
- BT656 接口 2 个 sensor 分别用到: sensor pad0/2
- 每个 device 都只有一个 channel Id0, 通过 MI_SYS 和后端绑定时只能用 channel id0

PortId	输入 Pixel	输出 Pixel	Crop	scaling	输出形式
0	按照 sensor 格式输入	1、 按照 sensor 格式输出；	不支持	不支持	1、 Dram 输出。 2、 一个 Device 支持

PortId	输入 Pixel	输出 Pixel	Crop	scaling	输出形式
		2、 将 YUV422 格式转成 12bit bayer 格式输出。			sensor 格式 realtime 输出到 MI_ISP, DeviceId 任意。 3、 一个 Device 支持 YUV422 格式 realtime 输出到 MI_SCL, DeviceId 任意。
1	只支持 YUV422 格式输入	只支持 12bit bayer 格式输出	支持	只支持 scaling down, height 有做 scaling down 时, width 最大等于 960。	只支持 Dram 输出。

1.2.2 Muffin 框图



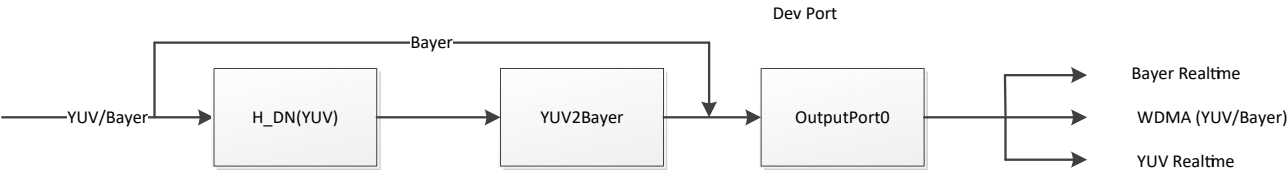


图 1-2: Muffin 框图

※ 注意

- 最大 32 个 Dev 可用。
- mipi 接口 8 个 sensor 分别用到：
linear mode: dev0/dev4/dev8/dev12, dev16/dev20/dev24/dev28
hdr frame mode: dev0/1, dev4/5, dev8/9 , dev12/13, dev16/17, dev20/21, dev24/25, dev28/29
hdr realtime mode: dev0/dev1, dev16/dev17
- BT656 接口 8 个 sensor 分别用到 pad0/pad2/pad1/pad3/pad4/pad6/pad5/pad7
- BT1120 接口 4 个 sensor 分别用到 pad0/pad1/pad4/pad5
- Lvds 接口 4 个 sensor 分别用到：
Linear mode: dev0/dev8,dev16/dev24
Hdr frame mode: dev0/dev1,dev8/dev9,dev16/dev17,dev24/dev25
Hdr realtime mode: dev0/dev1,dev16/dev17
- 每个 device 都只有一个 channel Id0, 通过 MI_SYS 和后端绑定时只能用 channel id0

PortId	输入 Pixel	输出 Pixel	Crop	scaling	输出形式
0	按照 sensor 格式输入	1、按照 sensor 格式输出； 2、将 YUV422 格式转成 12bit bayer 格式输出。	不支持	不支持	1、Dram 输出。 2、Dev0 支持 Bayer 格式 realtime 输出到 MI_ISP Dev0, Dev16 支持 Bayer 格式 realtime 输出到 MI_ISP Dev1 3、Dev8 支持 YUV422 格式 realtime 输出到 MI_SCL Dev2, Dev24 支持 YUV422 格式 realtime 输出到 MI_SCL Dev6

1.2.3 Mochi 框图

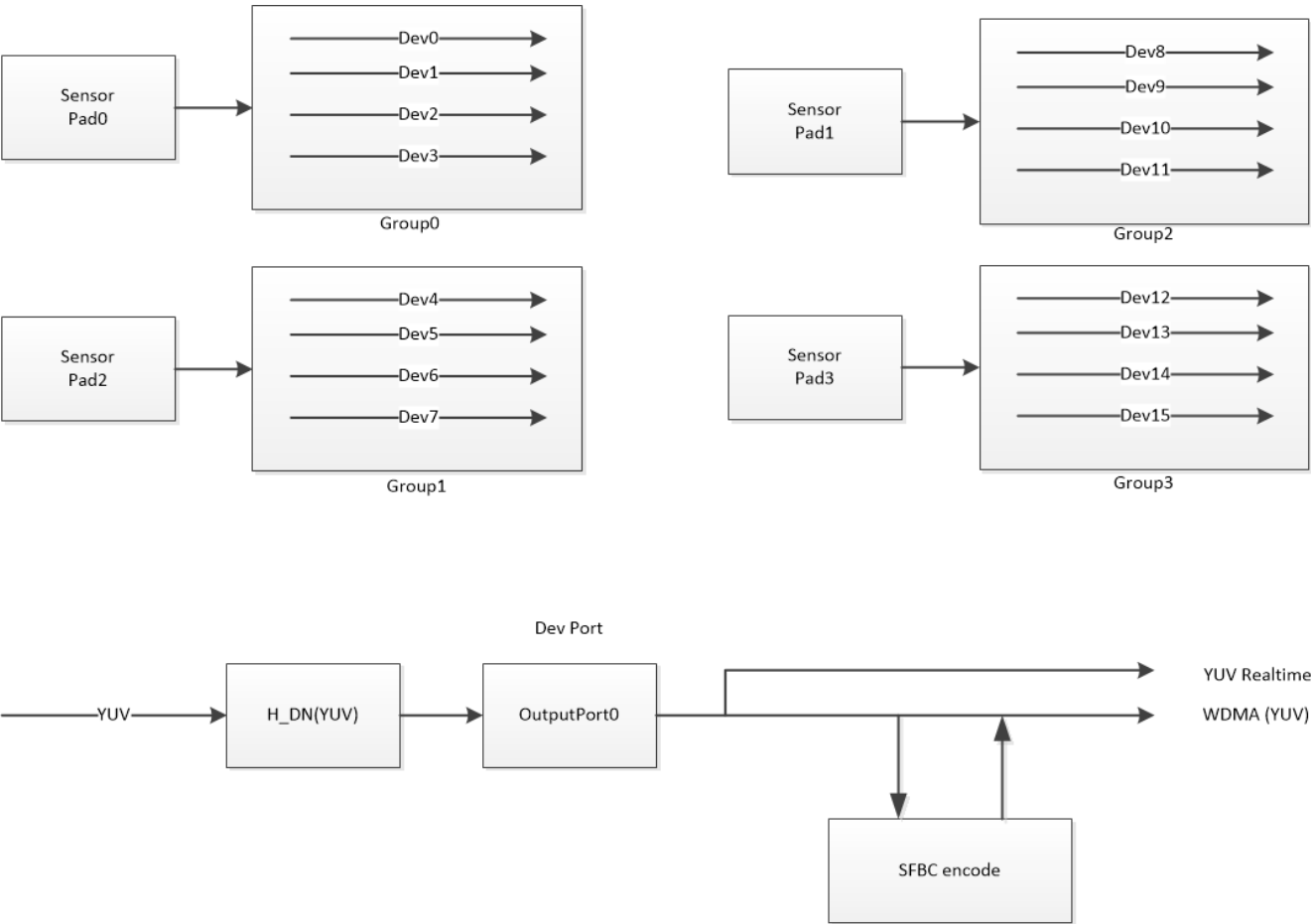


图 1-3: Mochi 框图

※ 注意

- 最大 16 个 Dev 可用。
- Mipi(YUV)接口 4 个 sensor 分别用到: pad0/pad2/pad1/pad3, 不支持 HDR, 类似 BT656, 1 个 pad 最多支持出 4 路。
- BT656 接口 4 个 sensor 分别用到 pad0/pad2/pad1/pad3。
- BT1120 接口 2 个 sensor 分别用到 pad0/pad1。
- 每个 device 都只有一个 channel Id0, 通过 MI_SYS 和后端绑定时只能用 channel id0

PortId	输入 Pixel	输出 Pixel	Crop	scaling	输出形式
0	只支持 YUV422 格式输入	只支持 YUV422 格式输出	不支持	不支持	1、Dram 输出。 2、Dev0 支持 yuv 格式 realtime 输出到 MI_ISP Dev0。

1.2.4 Maruko 框图

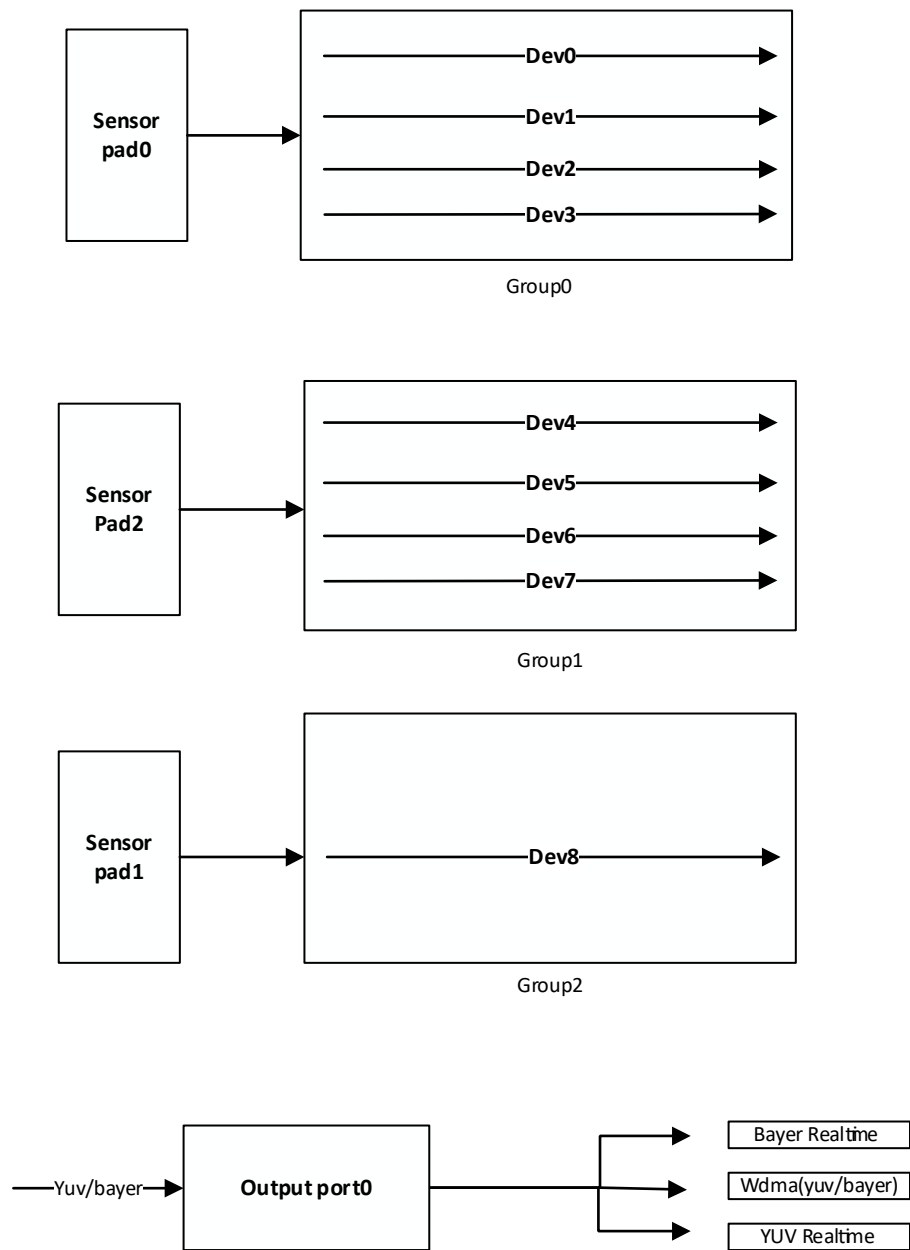


图 1-4: Maruko 框图

※ 注意

- 最大 9 个 Dev 可用。
- Mipi 接口 2 个 sensor 分别用到: pad0/pad2, 1 个 pad 最多支持出 4 路。Pad0 支持 4lane 或者拆成 2+2 lane (pad0+pad2)。
- BT656 只支持 sensor pad1, 只支持一路 sensor 信号。
- 不支持 BT1120。
- 每个 device 都只有一个 channel id0, 通过 MI_SYS 和后端绑定只能使用 channel id0。

PortId	输入 Pixel	输出 Pixel	Crop	scaling	输出形式
0	按照 sensor 格式输入	按照 sensor 格式输出	不支持	不支持	1. Dram 输出。 2. Dev0 支持 realtime 输出

					到 MI_ISP Dev0。 3. 支持 YUV422 格式 realtime 输出到 MI_SCL Dev2。
--	--	--	--	--	---

1.3. 关键字说明

- Group
群组，一个 sensor pad 中有可能混合多种信号，所以一个 sensor pad 需要一个 group 来对应接收。
- Dev
Group 中处理单独一种信号的设备。
- Port
Dev 上的输出端口。
- DI 模式
去隔行功能，将两场信号合成一场。

2. API 参考

该功能模块提供以下 API:

表 2-1: API 参考

API名	功能
MI_VIF_CreateDevGroup	创建Device 对应的Group
MI_VIF_CreateDevGroupExt	创建Group 并且设置一些额外属性
MI_VIF_DestroyDevGroup	销毁Group
MI_VIF_GetDevGroupAttr	获取Group 的属性
MI_VIF_SetDevAttr	设置设备属性
MI_VIF_GetDevAttr	获取设备属性
MI_VIF_EnableDev	启用设备
MI_VIF_DisableDev	禁用设备
MI_VIF_GetDevStatus	获取设备的状态
MI_VIF_SetOutputPortAttr	设置output 端口属性
MI_VIF_GetOutputPortAttr	获取output 端口属性
MI_VIF_EnableOutputPort	启用output 端口
MI_VIF_DisableOutputPort	禁用output 端口
MI_VIF_Query	查询 VIF 通道的Port中断计数、平均帧率等信息
MI_VIF_CallbackTask_Register	向 VIF 注册回调接口
MI_VIF_CallbackTask_UnRegister	向 VIF 反注册回调接口

2.1. MI_VIF_CreateDevGroup

➤ 功能

创建 Device 对应的 Group

➤ 语法

MI_S32 MI_VIF_CreateDevGroup([MI_VIF_GROUP](#) GroupId, [MI_VIF_GroupAttr_t](#) *pstGroupAttr)

➤ 形参

参数名称	描述	输入/输出
GroupId	Group ID	输入
pstGroupAttr	Group属性，静态属性。	输入

➤ 返回值

返回值 {
MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

➤ 注意

- MI VIF 跟后端模块同时存在 Realtime 和 FrameMode 连接时，例如 ISP Bayer Realitme，SCL Dev2 YUV Realitme 同时其他 Dev 有 Frame mode 连接，优先 Create Realtime 连接的 DevGroup 并 Bind 后端模块。
- 在 Create DevGroup 之前一定要确保 sensor 出流，即如果使用我们的 MI sensor，需要确保 MI_SNR_Enable，如果使用 user sensor 需要确保 user 自己的 sensor 已经完成初始化。

Group Id	包含 Device Id
0	Device0~3
1	Device4~7
2	Device8~11
3	Device12~15
4	Device16~19
5	Device20~23
6	Device24~27
7	Device28~31

➤ 举例

```
#define ST_MAX_VIF_DEV_PERGROUP (4)
#define ST_MAX_VIF_OUTPORT_NUM (2)
```

MI_VIF 初始化流程：

```
MI_S32 ST_VifModuleInit(MI_VIF_GROUP groupId)
{
    MI_VIF_DEV vifDev = 0;
    MI_VIF_PORT vifPort = 0;
    MI_SNR_PADID SnrPadId = 0;
    MI_U32 u32PlaneId = 0;
    MI_U16 vifDevIdPerGroup=0;

    MI_SNR_PADInfo_t stPad0Info;
```

```

MI_SNR_PlaneInfo_t stSnrPlane0Info;
memset(&stPad0Info, 0x0, sizeof(MI_SNR_PADInfo_t));
memset(&stSnrPlane0Info, 0x0, sizeof(MI_SNR_PlaneInfo_t));

MI_VIF_GroupAttr_t stGroupAttr;
memset(&stGroupAttr, 0x0, sizeof(MI_VIF_GroupAttr_t));

STCHECKRESULT(MI_SNR_GetPadInfo(SnrPadId , &stPad0Info));
STCHECKRESULT(MI_SNR_GetPlaneInfo(SnrPadId , u32PlaneId, &stSnrPlane0Info));

stGroupAttr.eIntfMode = E_MI_VIF_MODE_MIPI;
stGroupAttr.eWorkMode = E_MI_VIF_WORK_MODE_1MULTIPLEX;
stGroupAttr.eHDRType = E_MI_VIF_HDR_TYPE_OFF;
if(stGroupAttr.eIntfMode == E_MI_VIF_MODE_BT656)
    stGroupAttr.eClkEdge =
(MI_VIF_ClkEdge_e)stPad0Info.unIntfAttr.stBt656Attr.eClkEdge;
else
    stGroupAttr.eClkEdge = E_MI_VIF_CLK_EDGE_DOUBLE;

STCHECKRESULT(MI_VIF_CreateDevGroup(groupId , &stGroupAttr));

for(vifDevIdPerGroup=0; vifDevIdPerGroup< ST_MAX_VIF_DEV_PERGROUP;
vifDevIdPerGroup++)
{
    MI_VIF_DevAttr_t stVifDevAttr;
    memset(&stVifDevAttr, 0x0, sizeof(MI_VIF_DevAttr_t));

    vifDev = groupId *ST_MAX_VIF_DEV_PERGROUP+vifDevIdPerGroup;
    stVifDevAttr.stInputRect.u16X = stSnrPlane0Info.stCapRect.u16X;
    stVifDevAttr.stInputRect.u16Y = stSnrPlane0Info.stCapRect.u16Y;
    stVifDevAttr.stInputRect.u16Width = stSnrPlane0Info.stCapRect.u16Width;
    stVifDevAttr.stInputRect.u16Height = stSnrPlane0Info.stCapRect.u16Height;
    if(stSnrPlane0Info.eBayerId >= E_MI_SYS_PIXEL_BAYERID_MAX)
    {
        stVifDevAttr.eInputPixel = stSnrPlane0Info.ePixel;
    }
    else
        stVifDevAttr.eInputPixel =
(MI_SYS_PixelFormat_e)RGB_BAYER_PIXEL(stSnrPlane0Info.ePixPrecision,
stSnrPlane0Info.eBayerId);

    printf("setchnportattr (%d,%d,%d,%d) \n", stVifDevAttr.stInputRect.u16X,
stVifDevAttr.stInputRect.u16Y, stVifDevAttr.stInputRect.u16Width,

```

```

stVifDevAttr.stInputRect.u16Height);
    STCHECKRESULT(MI_VIF_SetDevAttr(vifDev, &stVifDevAttr));
    STCHECKRESULT(MI_VIF_EnableDev(vifDev));

    for(vifPort=0; vifPort< ST_MAX_VIF_OUTPORT_NUM; vifPort++)
    {
        MI_VIF_OutputPortAttr_t stVifPortInfo;
        memset(&stVifPortInfo, 0, sizeof(MI_VIF_OutputPortAttr_t));

        stVifPortInfo.stCapRect.u16X = stSnrPlane0Info.stCapRect.u16X;
        stVifPortInfo.stCapRect.u16Y = stSnrPlane0Info.stCapRect.u16Y;
        stVifPortInfo.stCapRect.u16Width = stSnrPlane0Info.stCapRect.u16Width;
        stVifPortInfo.stCapRect.u16Height = stSnrPlane0Info.stCapRect.u16Height;
        stVifPortInfo.stDestSize.u16Width = stSnrPlane0Info.stCapRect.u16Width;
        stVifPortInfo.stDestSize.u16Height = stSnrPlane0Info.stCapRect.u16Height;
        printf("sensor bayerid %d, bit mode %d \n", stSnrPlane0Info.eBayerId,
stSnrPlane0Info.ePixPrecision);
        if(stSnrPlane0Info.eBayerId >= E_MI_SYS_PIXEL_BAYERID_MAX)
        {
            stVifPortInfo.ePixFormat = stSnrPlane0Info.ePixel;
        }
        else
            stVifPortInfo.ePixFormat =
(MI_SYS_PixelFormat_e)RGB_BAYER_PIXEL(stSnrPlane0Info.ePixPrecision,
stSnrPlane0Info.eBayerId);
        stVifPortInfo.eFrameRate = E_MI_VIF_FRAMERATE_FULL;

        STCHECKRESULT(MI_VIF_SetOutputPortAttr(vifDev, vifPort, &stVifPortInfo));
        STCHECKRESULT(MI_VIF_EnableOutputPort(vifDev, vifPort));
    }
}
return MI_SUCCESS;
}

```

MI_VIF 去初始化流程:

```

MI_S32 ST_VifModuleUnInit(MI_VIF_GROUP groupId)
{
    MI_VIF_DEV vifDev = 0;
    MI_VIF_PORT vifPort=0;
    MI_U16 vifDevIdPerGroup=0;
    for(vifDevIdPerGroup=0; vifDevIdPerGroup< ST_MAX_VIF_DEV_PERGROUP;

```

```

vifDevIdPerGroup++)
{
    vifDev = groupId*ST_MAX_VIF_DEV_PERGROUP+vifDevIdPerGroup;

    for(vifPort=0; vifPort< ST_MAX_VIF_OUTPORT_NUM; vifPort++)
    {
        STCHECKRESULT(MI_VIF_DisableOutputPort(vifDev, vifPort));
    }
    STCHECKRESULT(MI_VIF_DisableDev(vifDev));
}

STCHECKRESULT(MI_VIF_DestroyDevGroup(groupId));

return MI_SUCCESS;
}

```

➤ 相关主题

[MI_VIF_DestroyDevGroup](#)

2.2. MI_VIF_CreateDevGroupExt

➤ 功能

创建 Group 并且设置一些额外属性。

➤ 语法

MI_S32 MI_VIF_CreateDevGroupExt(MI_VIF_GROUP GroupId, [MI_VIF_GroupAttr_t](#) *pstGroupAttr, [MI_VIF_GroupExtAttr_t](#) *pstGroupOptionExtAttr)

➤ 形参

参数名称	描述	输入/输出
GroupId	Group ID	输入
pstGroupAttr	Group 常规属性，静态属性	输入
pstGroupOptionExtAttr	Group 额外属性	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件: mi_vif_datatype.h、mi_vif.h

- 库文件: libmi_vif.a

➤ 注意

- VIF Group 和 sensor Pad, Device 和 PlaneId 有默认的对对应关系, 如果需要改变这个关系, 可以通过这个 api 设置 Group 接收对应 Sensor Pad 信号, Device 接收对应 Plane 信号。
- 在调用 MI_SNR_SetPlaneMode 后 VIF Group 和 sensor Pad, Device 和 PlaneId 对应关系会重置到默认情况, 所以应在 MI_SNR_SetPlaneMode 后调用 MI_VIF_CreateDevGroupExt。
- Tiramisu、Muffin、Mochi 系列芯片不支持改 Group 和 sensor pad 的对应关系。

➤ 举例

无。

➤ 相关主题

[MI_VIF_DestroyDevGroup](#)

2.3. MI_VIF_DestroyDevGroup

➤ 功能

销毁设备对应的 Group。

➤ 语法

MI_S32 MI_VIF_DestroyDevGroup([MI_VIF_GROUP](#) GroupId)

➤ 形参

参数名称	描述	输入/输出
GroupId	Group ID	输入

➤ 返回值

返回值 {
MI_SUCCESS (0) 成功。
非 0 失败, 详情参照[错误码](#)。

➤ 依赖

- 头文件: mi_vif_datatype.h、mi_vif.h
- 库文件: libmi_vif.a

➤ 注意

需要先使用 MI_VIF_DisableOutputPort 禁用掉 device 上所有输出端口, 再使用 MI_VIF_DisableDev 禁用掉 group 上所有 device。

➤ 举例

请参见 [MI_VIF_CreateDevGroup](#) 的举例。

- 相关主题
[MI_VIF_CreateDevGroup](#)

2.4. MI_VIF_GetDevGroupAttr

- 功能
获取 Group 的属性。
- 语法
MI_S32 MI_VIF_GetDevGroupAttr([MI_VIF_GROUP](#) GroupId, [MI_VIF_GroupAttr_t](#) *pstGroupAttr)

➤ 形参

参数名称	描述	输入/输出
GroupId	Group ID	输入
pstGroupAttr	Group属性	输出

- 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_vif_datatype.h、mi_vif.h
 - 库文件：libmi_vif.a
- 注意
无。
- 举例
无。
- 相关主题
[MI_VIF_CreateDevGroup](#)

2.5. MI_VIF_SetDevAttr

- 功能
设置 VIF 设备属性。
- 语法
MI_S32 MI_VIF_SetDevAttr(MI_VIF_DEV DevId, [MI_VIF_DevAttr_t](#) *pstDevAttr)
- 形参

参数名称	描述	输入/输出
DevId	VIF 设备号。	输入
pstDevAttr	VIF 设备属性指针，静态属性。	输入

➤ 返回值

返回值	MI_SUCCESS (0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

- 在调用前要保证 VIF 设备处于禁用状态。如果 VIF 设备已处于使能状态，可以使用 [MI_VIF_DisableDev](#) 来禁用设备。
- 参数 pstDevAttr 主要用来配置指定 VIF 设备的视频输入格式。
- eInputPixel 只支持 YUV422_YUYV/YVYU/UYVY/VYUY 和 bayer 格式，具体请参考[章节 1.2](#)，根据 chip 规格设置。

➤ 举例

请参见 [MI_VIF_CreateDevGroup](#) 的举例。

➤ 相关主题

[MI_VIF_GetDevAttr](#)

2.6. MI_VIF_GetDevAttr

➤ 功能

获取 VIF 设备属性。

➤ 语法

MI_S32 MI_VIF_GetDevAttr(MI_VIF_DEV DevId, [MI_VIF_DevAttr_t](#) *pstDevAttr)

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号。	输入
pstDevAttr	VIF 设备属性指针。	输出

- 返回值
 - 返回值
 - MI_SUCCESS (0) 成功。
 - 非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_vif_datatype.h、mi_vif.h
 - 库文件：libmi_vif.a
- 注意
 - 无。
- 举例
 - 无。
- 相关主题
 - [MI_VIF_SetDevAttr](#)

2.7. MI_VIF_EnableDev

- 功能
 - 启用 VIF 设备。
- 语法
 - MI_S32 MI_VIF_EnableDev(MI_VIF_DEV DevId);

- 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入

- 返回值
 - 返回值
 - MI_SUCCESS (0) 成功。
 - 非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_vif_datatype.h、mi_vif.h
 - 库文件：libmi_vif.a

※ 注意

- 启用前必须已经设置设备属性，否则返回失败。
- 可重复启用，不返回失败。

➤ 举例

请参见 [MI_VIF_SetDevAttr](#) 的举例。

➤ 相关主题

[MI_VIF_DisableDev](#)

2.8. MI_VIF_DisableDev

➤ 功能

禁用 VIF 设备。

➤ 语法

```
MI_S32 MI_VIF_DisableDev(MI_VIF_DEV DevId);
```

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入

➤ 返回值


 MI_SUCCESS (0) 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

- 必须先禁用所有设备上的输出端口后，才能禁用 VIF 设备。
- 可重复禁用，不返回失败。

➤ 举例

请参见 [MI_VIF_CreateDevGroup](#) 的举例。

➤ 相关主题

[MI_VIF_EnableDev](#)

2.9. MI_VIF_GetDevStatus

➤ 功能

获取设备的状态。

➤ 语法

MI_S32 MI_VIF_GetDevStatus(MI_VIF_DEV DevId, [MI_VIF_DevStatus_t](#) *pstVifDevStatus)

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
pstVifDevStatus	设备状态	输出

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

2.10. MI_VIF_SetOutputPortAttr

➤ 功能

设置 VIF 输出端口属性。

➤ 语法

MI_S32 MI_VIF_SetOutputPortAttr(MI_VIF_DEV DevId, MI_VIF_PORT PortId,
[MI_VIF_OutputPortAttr_t](#) *pstAttr);

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
PortId	端口号	输入
pstAttr	VIF 设备Port属性指针	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

➤ 注意

- 默认情况下，使用 MI_VIF_SetOutputPortAttr 接口的目的是设置端口属性,如 CapSize 、 DestSize、FrameRate 等等。
- Port0 不支持 crop 和 scaling, 所以 cap 和 dest size 必须和 MI_VIF_SetDevAttr 时的 input size 相等。
- Tiramisu 系列芯片支持 port1, port1 输入只支持 YUV 格式，所以只有 YUV sensor 才可以使用 port1, 支持 crop 和 scaling down，输出只支持 12bit bayer 格式。
- Tiramisu 系列芯片支持 port1, Port1 height 有做 scaling down 时，输出 width 最大等于 960。
- 目前不支持 eFrameRate 设置，预留参数。
- Tiramisu 、Muffin 系列芯片：port0 支持输出 12bit bayer 格式，可以送进 MI_ISP 做 3DNR。
- Mochi 芯片：支持输出 YUV422 格式，可以送进 MI_ISP 做 3DNR。

芯片	Pixel 要求	支持压缩模式	注意
Tiramisu	Bayer 10/12bit	E_MI_SYS_COMPRESS_MODE_TO_8BIT	无
Muffin	Bayer 10/12bit	E_MI_SYS_COMPRESS_MODE_TO_8BIT	无
Mochi	YUV422 UYVY	E_MI_SYS_COMPRESS_MODE_TO_6BIT E_MI_SYS_COMPRESS_MODE_SFBC0	DI 模式不支持 SFBC0 压缩
Maruko	Bayer 10/12bit	E_MI_SYS_COMPRESS_MODE_TO_8BIT	无

➤ 举例

请参见 [MI_VIF_CreateDevGroup](#) 的举例。

➤ 相关主题

[MI_VIF_GetOutputPortAttr](#)

2.11. MI_VIF_GetOutputPortAttr

➤ 功能

获取 VIF 输出端口属性。

➤ 语法

MI_S32 MI_VIF_GetOutputPortAttr(MI_VIF_DEV DevId, MI_VIF_PORT PortId,
[MI_VIF_OutputPortAttr_t](#) *pstAttr)

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
PortId	Port号	输入
pstAttr	VIF 设备Port属性指针	输出

➤ 返回值

返回值

- MI_SUCCESS (0) 成功。
- 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

无。

➤ 举例

无。

➤ 相关主题

[MI_VIF_SetOutputPortAttr](#)

2.12. MI_VIF_EnableOutputPort

➤ 功能

启用 VIF 输出端口。

➤ 语法

MI_S32 MI_VIF_EnableOutputPort(MI_VIF_DEV DevId, MI_VIF_PORT PortId)

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
PortId	Port号	输入

➤ 返回值

返回值	MI_SUCCESS (0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

- 必须先设置通道属性，且通道所绑定的 VIF 设备必须使能。
- 可重复启用 VIF 通道，不返回失败。

➤ 举例

请参见 [MI_VIF_SetDevAttr](#) 的举例。

➤ 相关主题

[MI_VIF_DisableOutputPort](#)

2.13. MI_VIF_DisableOutputPort

➤ 功能

禁用 VIF 输出端口。

➤ 语法

MI_S32 MI_VIF_DisableOutputPort(MI_VIF_DEV DevId, MI_VIF_PORT PortId)

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
PortId	Port号	输入

➤ 返回值

返回值	MI_SUCCESS (0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

- 禁用 VIF 通道 Port 后，此 VIF 通道 Port 停止采集视频输入数据，如果已经绑定后端，则后端不会再接收到视频图像。
- 可重复禁用 VIF 通道 Port，不返回失败。

➤ 举例

请参见 [MI_VIF_CreateDevGroup](#) 的举例。

➤ 相关主题

[MI_VIF_EnableOutputPort](#)

2.14. MI_VIF_Query

➤ 功能

查询 VIF 通道的中断计数、平均帧率等信息。

➤ 语法

MI_S32 MI_VIF_Query(MI_VIF_DEV DevId, MI_VIF_PORT PortId, [MI_VIF_DevPortStat_t](#) *pstStat)

➤ 形参

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
PortId	Port 口	输入
pstStat	通道信息结构体指针	输出

➤ 返回值

{ 返回值
 MI_SUCCESS (0) 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_vif_datatype.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

- 该接口可查询中断计数、通道使能状态、平均帧率、中断丢失数、获取 VB 失败次数、图像宽高等信息。
- 通过该接口获取到的帧率是每 1 秒钟的平均帧率，即 VIF 会每隔一秒统计一次平均帧率，该值并不精确，会有些波动。
- 用户可通过该接口查询中断丢失数，如果该数值一直在增加，说明 VIF 工作出现异常。

- Tiramisu 系列以后的芯片才支持该接口。

- 举例
无。
- 相关主题
无。

2.15. MI_VIF_CallbackTask_Register

- 描述
向 VIF 注册回调接口。
- 语法
MI_S32 MI_VIF_CallbackTask_Register(MI_VIF_DEV DevId, [MI_VIF_CallbackParam_t](#) *pstCallbackParam);

- 参数

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
pstCallbackParam	回调参数	输入

- 返回值
返回值 { MI_SUCCESS (0) 成功。
非 0 失败，参照[错误码](#)。

- 依赖
 - 头文件：mi_common.h、mi_vif.h
 - 库文件：libmi_vif.a

- ※ 注意
 - 该接口目前只支持 kernel mode 调用
 - 和 [MI_VIF_CallbackTask_UnRegister](#) 成对调用

- 举例

```
MI_S32 _mi_vif_framestart1(MI_U64 u64Data)
{
    DBG_ERR("DATA %llu \n", u64Data);
    return 0;
}
MI_S32 _mi_vif_framestart2(MI_U64 u64Data)
{
    DBG_ERR("DATA %llu \n", u64Data);
}
```

```
    return 0;
}

static MS_S32 _mi_vif_testRegVifCallback(void)
{
    MI\_VIF\_CallbackParam\_t stCallbackParam1;
    MI\_VIF\_CallbackParam\_t stCallbackParam2;
    MI_VIF_DEV u32DevId = 0;
    memset(&stCallbackParam1, 0x0, sizeof(MI_VIF_CallbackParam_t));
    memset(&stCallbackParam2, 0x0, sizeof(MI_VIF_CallbackParam_t));
    stCallbackParam1.eCallbackMode = E_MI_VIF_CALLBACK_ISR;
    stCallbackParam1.eIrqType = E_MI_VIF_IRQ_FRAMESTART;
    stCallbackParam1.pfnCallbackFunc = _mi_vif_framestart1;
    stCallbackParam1.u64Data = 11;
    MI\_VIF\_CallbackTask\_Register(u32DevId,&stCallbackParam1);
    stCallbackParam2.eCallbackMode = E_MI_VIF_CALLBACK_ISR;
    stCallbackParam2.eIrqType = E_MI_VIF_IRQ_FRAMESTART;
    stCallbackParam2.pfnCallbackFunc = _mi_vif_framestart2;
    stCallbackParam2.u64Data = 22;
    MI\_VIF\_CallbackTask\_Register(u32DevId,&stCallbackParam2);
    return 0;
}

static MS_S32 _mi_vif_testUnRegVifCallback(void)
{
    MI\_VIF\_CallbackParam\_t stCallbackParam1;
    MI\_VIF\_CallbackParam\_t stCallbackParam2;

    MI_VIF_DEV u32DevId = 0;
    memset(&stCallbackParam1, 0x0, sizeof(MI_VIF_CallbackParam_t));
    memset(&stCallbackParam1, 0x0, sizeof(MI_VIF_CallbackParam_t));
    stCallbackParam1.eCallbackMode = E_MI_VIF_CALLBACK_ISR;
    stCallbackParam1.eIrqType = E_MI_VIF_IRQ_FRAMESTART;
    stCallbackParam1.pfnCallbackFunc = _mi_vif_framestart1;
    stCallbackParam1.u64Data = 33;
    MI\_VIF\_CallbackTask\_UnRegister(u32DevId,&stCallbackParam1);
    stCallbackParam2.eCallbackMode = E_MI_VIF_CALLBACK_ISR;
    stCallbackParam2.eIrqType = E_MI_VIF_IRQ_FRAMESTART;
    stCallbackParam2.pfnCallbackFunc = _mi_vif_framestart2;
    stCallbackParam2.u64Data = 44;
    MI\_VIF\_CallbackTask\_UnRegister(u32DevId,&stCallbackParam2);
    return 0;
}
```

[MI_VIF_CallbackTask_UnRegister](#)

2.16. [MI_VIF_CallbackTask_UnRegister](#)

➤ 描述
向 VIF 反注册回调接口。

➤ 语法
`MI_S32 MI_VIF_CallbackTask_UnRegister(MI_VIF_DEV DevId, MI\_VIF\_CallbackParam\_t *pstCallbackParam);`

➤ 参数

参数名称	描述	输入/输出
DevId	VIF 设备号	输入
pstCallbackParam	回调参数	输入

➤ 返回值
返回值 { MI_SUCCESS (0) 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_common.h、mi_vif.h
- 库文件：libmi_vif.a

※ 注意

- 该接口目前只支持 kernel mode 调用
- 和 [MI_VIF_CallbackTask_Register](#) 成对调用

➤ 举例
见 [MI_VIF_CallbackTask_Register](#) 举例

➤ 相关主题
[MI_VIF_CallbackTask_Register](#)

3. VIF 数据类型

视频输入相关数据类型定义如下：

表 3-2: VIF 数据类型

数据类型	描述
MI_VIF_GROUP	定义设备群组Id
MI_VIF_IntfMode_e	定义视频输入设备的接口模式
MI_VIF_WorkMode_e	定义视频设备的复合工作模式
MI_VIF_FrameRate_e	定义视频设备输出fps和输入fps的关系
MI_VIF_ClkEdge_e	定义视频设备接收的时钟类型
MI_VIF_HDRType_e	定义视频设备 HDR 类型
MI_VIF_MclkSource_e	定义给 Sensor 输入驱动Clock 类型
MI_VIF_GroupIdMask_e	定义设备群组mask
MI_VIF_SNRPad_e	定义 SensorPad Id
MI_VIF_GroupAttr_t	定义设备群组属性
MI_VIF_GroupExtAttr_t	定义设备群组额外属性
MI_VIF_DevAttr_t	定义视频输入设备的属性。
MI_VIF_OutputPortAttr_t	定义 VIF输出端口属性。
MI_VIF_DevPortStat_t	定义VIF输出端口信息结构体
MI_VIF_DevStatus_t	定义VIF设备当前状态
MI_VIF_CALLBK_FUNC	定义回调函数类型
MI_VIF_CallbackMode_e	定义回调模式
MI_VIF_IrqType_e	定义硬件中断类型
MI_VIF_CallbackParam_t	定义回调参数

3.1. MI_VIF_GROUP

➤ 说明

定义设备群组 ID。

➤ 定义

```
typedef MI_U32 MI_VIF_GROUP;
```

※ 注意事项

- 无。
- 相关数据类型及接口
[MI_VIF_CreateDevGroup](#)

3.2. MI_VIF_IntfMode_e

- 说明
定义视频设备的接口模式。

- 定义

```
typedef enum
{
    E_MI_VIF_MODE_BT656 = 0,
    E_MI_VIF_MODE_DIGITAL_CAMERA,
    E_MI_VIF_MODE_BT1120_STANDARD,
    E_MI_VIF_MODE_BT1120_INTERLEAVED,
    E_MI_VIF_MODE_MIPI,
    E_MI_VIF_MODE_LVDS,
    E_MI_VIF_MODE_NUM
} MI_VIF_IntfMode_e;
```

- 成员

成员名称	描述
E_MI_VIF_MODE_BT656	输入数据的协议符合标准 BT.656 协议，端口数据输入模式为亮度色度复合模式，分量模式为单分量。
E_MI_VIF_MODE_DIGITAL_CAMERA	输入数据的协议为 Digital camera 协议，端口数据输入模式为亮度色度复合模式，分量模式为单分量。
E_MI_VIF_MODE_BT1120_STANDARD	输入数据的协议符合标准 BT.1120 协议（ BT.656+ 双分量），端口数据输入模式为亮度色度分离模式，分量模式为双分量。
E_MI_VIF_MODE_BT1120_INTERLEAVED	输入数据的协议符合 BT.1120 interleaved 模式，端口数据输入模式为亮度色度分离模式，分量模式为双分量。
E_MI_VIF_MODE_MIPI	输入数据符合 MIPI 协议
E_MI_VIF_MODE_LVDS	输入数据符合 LVDS 协议

- ※ 注意事项
可以通过 **MI_SNR_GetPadInfo** 中的 **eIntfMode** 获取当前接口模式。

- 相关数据类型及接口
[MI_VIF_GroupAttr_t](#)

3.3. MI_VIF_WorkMode_e

➤ 说明

定义视频设备的复合工作模式。

➤ 定义

```
typedef enum
{
    /* BT656 multiple ch mode */
    E_MI_VIF_WORK_MODE_1MULTIPLEX,
    E_MI_VIF_WORK_MODE_2MULTIPLEX,
    E_MI_VIF_WORK_MODE_4MULTIPLEX,
    E_MI_VIF_WORK_MODE_MAX
} MI_VIF_WorkMode_e;
```

➤ 成员

成员名称	描述
E_MI_VIF_WORK_MODE_1MULTIPLEX	1 路复合工作模式。
E_MI_VIF_WORK_MODE_2MULTIPLEX	2 路复合工作模式。
E_MI_VIF_WORK_MODE_4MULTIPLEX	4 路复合工作模式。

※ 注意事项

例如 E_MI_VIF_WORK_MODE_4MULTIPLEX，代表 Group 对应的 Sensor Pad 中有 4 路混合视频信号，如果 SensorPad 中混合信号数量，和 Group 的复合工作模式不匹配，Group 会采集信号异常。

➤ 相关数据类型及接口

[MI_VIF_GroupAttr_t](#)

3.4. MI_VIF_FrameRate_e

➤ 说明

定义视频设备输出 fps 和输入 fps 的关系。

➤ 定义

```
typedef enum
{
    E_MI_VIF_FRAMERATE_FULL = 0,
    E_MI_VIF_FRAMERATE_HALF,
    E_MI_VIF_FRAMERATE_QUARTR,
    E_MI_VIF_FRAMERATE_OCTANT,
    E_MI_VIF_FRAMERATE_THREE_QUARTERS,
    E_MI_VIF_FRAMERATE_NUM,
} MI_VIF_FrameRate_e;
```

➤ 成员

成员名称	描述
E_MI_VIF_FRAMERATE_FULL	源和目标1:1输出。
E_MI_VIF_FRAMERATE_HALF	源和目标2:1输出。
E_MI_VIF_FRAMERATE_QUARTER	源和目标4:1输出。
E_MI_VIF_FRAMERATE_OCTANT	源和目标8:1输出。
E_MI_VIF_FRAMERATE_THREE_QUARTER	源和目标4:3输出。

※ 注意事项

该功能为预留功能，暂不支持，如果需要控制帧率输出，可以通过 MI_SYS_BindChnPort2 来设置。

➤ 相关数据类型及接口

[MI_VIF_OutputPortAttr_t](#)

3.5. MI_VIF_ClkEdge_e

➤ 说明

定义视频设备接收的时钟类型。

➤ 定义

```
typedef enum
{
    E_MI_VIF_CLK_EDGE_SINGLE_UP = 0,
    E_MI_VIF_CLK_EDGE_SINGLE_DOWN,
    E_MI_VIF_CLK_EDGE_DOUBLE,
    E_MI_VIF_CLK_EDGE_NUM
} MI_VIF_ClkEdge_e;
```

➤ 成员

成员名称	描述
E_MI_VIF_CLK_EDGE_SINGLE_UP	时钟单沿模式，且 VIF 设备在上升沿采样。
E_MI_VIF_CLK_EDGE_SINGLE_DOWN	时钟单沿模式，且 VIF 设备在下降沿采样。
E_MI_VIF_CLK_EDGE_DOUBLE	前端送过来双沿数据时，VIF 进行双沿采样。

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_VIF_GroupAttr_t](#)

3.6. MI_VIF_HDRType_e

➤ 说明

定义视频设备 HDR 类型。

➤ 定义

```
typedef enum
{
    E_MI_VIF_HDR_TYPE_OFF,
    E_MI_VIF_HDR_TYPE_VC,    //virtual channel mode HDR,vc0->long, vc1->short
    E_MI_VIF_HDR_TYPE_DOL,
    E_MI_VIF_HDR_TYPE_EMBEDDED, //compressed HDR mode
    E_MI_VIF_HDR_TYPE_LI,    //Line interlace HDR
    E_MI_VIF_HDR_TYPE_MAX
} MI_VIF_HDRType_e;
```

➤ 成员

成员名称	描述
E_MI_VIF_HDR_TYPE_OFF	不开HDR
E_MI_VIF_HDR_TYPE_VC	virtual channel mode HDR,vc0->long, vc1->short
E_MI_VIF_HDR_TYPE_DOL	Digital Overlap High Dynamic Range
E_MI_VIF_HDR_TYPE_EMBEDDED	compressed HDR mode
E_MI_VIF_HDR_TYPE_LI	Line interlace HDR

※ 注意事项

- hdr type 和 sensor 相关， 可以通过 MI_SNR_GetPadInfo 的 eHDRMode 获取当前 sensor 支持的 HDR type。
- 实现 HDR 功能需要 MI_VIF 和 MI_ISP 模块设置相同的 HDR 类型。

➤ 相关数据类型及接口

[MI_VIF_GroupAttr_t](#)

3.7. MI_VIF_MclkSource_e

➤ 说明

设置 Senor 时钟驱动。

➤ 定义

```
typedef enum
{
    E_MI_VIF_MCLK_12MHZ,
    E_MI_VIF_MCLK_18MHZ,
    E_MI_VIF_MCLK_27MHZ,
    E_MI_VIF_MCLK_36MHZ,
    E_MI_VIF_MCLK_54MHZ,
    E_MI_VIF_MCLK_108MHZ,
    E_MI_VIF_MCLK_MAX
}MI_VIF_MclkSource_e;
```

➤ 成员

成员名称	描述
E_MI_VIF_MCLK_12MHZ	12M clk 类型
E_MI_VIF_MCLK_18MHZ	18M clk 类型
E_MI_VIF_MCLK_27MHZ	27M clk 类型
E_MI_VIF_MCLK_36MHZ	36M clk 类型
E_MI_VIF_MCLK_54MHZ	54M clk 类型
E_MI_VIF_MCLK_108MHZ	108M clk 类型

※ 注意事项

- 有 sensor driver 的情况下，优先从 sensor driver 中设置 mclk。
- 如果没有 sensor driver 并且需要主控芯片输出 mclk 才通过该参数设置。

➤ 相关数据类型及接口

[MI_VIF_GroupAttr_t](#)

3.8. MI_VIF_GroupIdMask_e

➤ 说明

设备群组 mask。

➤ 定义

```
typedef enum
{
    E_MI_VIF_GROUPMASK_ID0 = 0x0001,
    E_MI_VIF_GROUPMASK_ID1 = 0x0002,
    E_MI_VIF_GROUPMASK_ID2 = 0x0004,
    E_MI_VIF_GROUPMASK_ID3 = 0x0008,
    E_MI_VIF_GROUPMASK_ID4 = 0x0010,
    E_MI_VIF_GROUPMASK_ID5 = 0x0020,
    E_MI_VIF_GROUPMASK_ID6 = 0x0040,
    E_MI_VIF_GROUPMASK_ID7 = 0x0080,
    E_MI_VIF_GROUPMASK_ID_MAX = 0xffff
} MI_VIF_GroupIdMask_e;
```

※ 注意事项

在多 sensor 拼接场景中，需要两个 sensor 图像同步输出，此时只需要创建一个 group，将这两个 group id mask 在一起即可。

➤ 相关数据类型及接口

[MI_VIF_GroupAttr_t](#)

3.9. MI_VIF_SNRPad_e

➤ 说明

定义 SensorPad Id。

➤ 定义

```
typedef enum
{
    E_MI_VIF_SNRPAD_ID_0 = 0,
    E_MI_VIF_SNRPAD_ID_1 = 1,
    E_MI_VIF_SNRPAD_ID_2 = 2,
    E_MI_VIF_SNRPAD_ID_3 = 3,
    E_MI_VIF_SNRPAD_ID_4 = 4,
    E_MI_VIF_SNRPAD_ID_5 = 5,
    E_MI_VIF_SNRPAD_ID_6 = 6,
    E_MI_VIF_SNRPAD_ID_7 = 7,
    E_MI_VIF_SNRPAD_ID_MAX,
    E_MI_VIF_SNRPAD_ID_NA = 0xFF,
}MI_VIF_SNRPad_e;
```

➤ 成员

成员名称	描述
E_MI_VIF_SNRPAD_ID_0	对应硬件设备Sensor0
E_MI_VIF_SNRPAD_ID_1	对应硬件设备Sensor1
E_MI_VIF_SNRPAD_ID_2	对应硬件设备Sensor2
E_MI_VIF_SNRPAD_ID_3	对应硬件设备Sensor3
E_MI_VIF_SNRPAD_ID_4	对应硬件设备Sensor4
E_MI_VIF_SNRPAD_ID_5	对应硬件设备Sensor5
E_MI_VIF_SNRPAD_ID_6	对应硬件设备Sensor6
E_MI_VIF_SNRPAD_ID_7	对应硬件设备Sensor7
E_MI_VIF_SNRPAD_ID_MAX	超过最大Sensor Num
E_MI_VIF_SNRPAD_ID_NA	无效sensor id

※ 注意事项

- 在默认情况下是 VIF Group0 对应 Sensor0, Group2 对应 Sensor1, group6 对应 Sensor5。
- Tiramisu、Mochi 只有 4 个 pad(0-3), Muffin 则有 8 个 pad(0-7)。参考[章节 1.2](#)。

➤ 相关数据类型及接口

[MI_VIF_GroupExtAttr_t](#)

[MI_VIF_DevStatus_t](#)

3.10. MI_VIF_GroupAttr_t

➤ 说明

定义 Group 群组属性。

➤ 定义

```
typedef struct MI_VIF_GroupAttr_s
{
    MI_VIF_IntfMode_e    eIntfMode;
    MI_VIF_WorkMode_e    eWorkMode;
    MI_VIF_HDRTYPE_e     eHDRTYPE;
    MI_VIF_ClkEdge_e     eClkEdge; //BT656
    MI_VIF_MclkSource_e   eMclk;
    MI_SYS_FrameScanMode_e eScanMode;
    MI_U32    u32GroupStitchMask; //multi vif dev bitmask by MI_VIF_GroupIdMask_e
} MI_VIF_GroupAttr_t;
```

➤ 成员

成员名称	描述
MI_VIF_IntfMode_e	接口模式。
MI_VIF_WorkMode_e	工作模式。
MI_VIF_HDRTYPE_e	HDR类型
MI_VIF_ClkEdge_e	时钟边沿模式（上升沿采样、下降沿采样、双沿采样）。
MI_VIF_MclkSource_e	输出给sensor的时钟类型
MI_SYS_FrameScanMode_e	输入扫描模式（逐行、隔行）
u32GroupStitchMask	多个GroupId缝合输出, 由MI_VIF_GroupIdMask_e bitmask 组成。

※ 注意事项

Tiramisu、Muffin 系列芯片 eScanMode 只支持 E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE。

➤ 相关数据类型及接口

[MI_VIF_CreateDevGroup](#)
[MI_VIF_CreateDevGroupExt](#)
[MI_VIF_GetDevGroupAttr](#)

3.11. MI_VIF_GroupExtAttr_t

➤ 说明

定义 SensorPad Id。

➤ 定义

```
typedef struct MI_VIF_GroupExtAttr_s
{
    MI_VIF_SNRPad_e eSensorPadID; //sensor Pad id
    MI_U32 u32PlaneID[MI_VIF_MAX_GROUP_DEV_CNT]; //For HDR,0 is short exposure, 1 is long exposure
} MI_VIF_GroupExtAttr_t;
```

➤ 成员

成员名称	描述
eSensorPadID	硬件设备Sensor号
u32PlaneID	PlaneId, for HDR 1 长曝, 0 短曝, for liner 为0xff.

※ 注意事项

默认情况下按照 1.2 章节框架图 sensor Pad 和 Vif Group 之间的映射。

➤ 相关数据类型及接口

[MI_VIF_CreateDevGroupExt](#)

3.12. MI_VIF_DevAttr_t

➤ 说明

定义视频输入设备的属性。

➤ 定义

```
typedef struct MI_VIF_DevAttr_s
{
    MI_SYS_PixelFormat_e eInputPixel;
    MI_SYS_WindowRect_t stInputRect;
    MI_SYS_FieldType_e eField;
    MI_BOOL bEnH2T1PMode;
} MI_VIF_DevAttr_t;
```

➤ 成员

成员名称	描述
eInputPixel	设备输入像素格式。
stInputRect	设备采集输入范围。
eField	帧场选择, 只用于隔行模式, 建议捕获单场时选择捕获底场。逐行模式时, 该项必须设置为 E_MI_SYS_FIELDTYPE_NONE。
bEnH2T1PMode	使能采集水平方向缩小一半功能。

※ 注意事项

- 如果设备是 bayer 格式, pixel format 设置形式如下:
stVifDevAttr.eInputPixel =
(MI_SYS_PixelFormat_e)RGB_BAYER_PIXEL(stSnrPlane0Info.ePixPrecision,
stSnrPlane0Info.eBayerId);
- bEnH2T1Pmode 参数只有输入是 YUV 格式时 才可以使用, 非 YUV 格式使用会返回 err。

➤ 相关数据类型及接口

[MI_VIF_SetDevAttr](#)

[MI_VIF_GetDevAttr](#)

3.13. MI_VIF_OutputPortAttr_t

➤ 说明

定义 VIF 通道 Port 属性。

➤ 定义

```
typedef struct MI_VIF_OutputPortAttr_s
{
    MI_SYS_WindowRect_t      stCapRect;
    MI_SYS_WindowSize_t      stDestSize;
    MI_SYS_PixelFormat_e      ePixFormat;
    MI_VIF_FrameRate_e        eFrameRate;
    MI_SYS_CompressMode_e      eCompressMode;
} MI_VIF_OutputPortAttr_t;
```

➤ 成员

成员名称	描述
stCapRect	捕获区域起始坐标(相对于设备图像的大小)与宽高。
stDestSize	目标图像大小。必须配置，且大小不应该超出外围 ADC 输出图像的大小范围，否则可能导致VIF 硬件工作异常。
ePixFormat	像素存储格式支持，可支持YUV422 packet/bayer。
eFrameRate	目标帧率和输入帧率的比值关系。如果不进行帧率控制，则该值设置为0。可以按照1:1,2:1,4:1,8:1,4:3等比例输出。该功能为预留功能，暂不支持，如果需要控制帧率输出，可以通过MI_SYS_BindChnPort2 来设置。
eCompressMode	图像压缩模式，只有在vif与isp绑定模式为framemode才能打开，节省buffer，不设置默认值为0不压缩。 Muffin可以设置成E_MI_SYS_COMPRESS_MODE_TO_8BIT，打开FBC。 Mochi 可以设置成 E_MI_SYS_COMPRESS_MODE_TO_6BIT (for YUV FBC)或者E_MI_SYS_COMPRESS_MODE_SFBC0 (SFBC)。

➤ 相关数据类型及接口

[MI_VIF_SetOutputPortAttr](#)

[MI_VIF_GetOutputPortAttr](#)

3.14. MI_VIF_DevPortStat_t

➤ 说明

VIF 通道信息结构体。

➤ 定义

```
typedef struct MI_VIF_DevPortStat_s
{
    MI_BOOL bEnable;
```

```
MI_U32 u32IntCnt;
MI_U32 u32FrameRate;
MI_U32 u32LostInt;
MI_U32 u32VbFail;
MI_U32 u32PicWidth;
MI_U32 u32PicHeight;
} MI_VIF_DevPortStat_t;
```

➤ 成员

成员名称	描述
bEnable	通道是否使能。
u32IntCnt	中断计数。
u32FrameRate	每 1 秒的平均帧率，该值不一定精确。
u32LostInt	中断丢失计数。
u32VbFail	获取 VB 失败计数。
u32PicWidth	图像宽度。
u32PicHeight	图像高度。

※ 注意事项

- 结构体的中断计数，可用于无中断检测。
- 该结构体的帧率是每 1 秒钟的平均帧率，即 VIF 会每隔 1 秒统计一次平均帧率，该值并不精确。
- 如果查询到该结构体的中断丢失计数一直在增加，说明 VIF 工作出现异常。

➤ 相关数据类型及接口

[MI_VIF_Query](#)

3.15. MI_VIF_DevStatus_t

➤ 说明

VIF Dev 当前状态

➤ 定义

```
typedef struct MI_VIF_VIFDevStatus_s
{
    MI\_VIF\_GROUP GroupId;
    MI_BOOL bGroupCreated;
    MI_U32 bDevEn;
    MI\_VIF\_SNRPad\_e eSensorPadID;
    MI_U32 u32PlaneID;
} MI_VIF_DevStatus_t;
```

➤ 成员

成员名称	描述
GroupId	Device 所属Group number

bGroupCreated	Group 当前Create 状态
bDevEn	VIF Dev 当前使能状态
eSensorPadID	VIF Dev 绑定的SensorPad
u32PlaneID	VIF Dev 绑定的PlaneId

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_VIF_GetDevStatus](#)

3.16. MI_VIF_CALLBACK_FUNC

➤ 说明
定义回调函数类型

➤ 定义
typedef MI_S32 (*MI_VIF_CALLBACK_FUNC)(MI_U64 u64Data);

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_VIF_CallbackParam_t](#)

3.17. MI_VIF_CallbackMode_e

➤ 说明
定义回调模式。

➤ 定义
typedef enum
{
 E_MI_VIF_CALLBACK_ISR,
 E_MI_VIF_CALLBACK_MAX,
} MI_VIF_CallbackMode_e;

➤ 成员

成员名称	描述
E_MI_VIF_CALLBACK_ISR	硬件中断模式回调
E_MI_VIF_CALLBACK_MAX	回调模式最大值

※ 注意事项
目前只支持 **ISR** 回调模式。

- 相关数据类型及接口

[MI_VIF_CallbackParam_t](#)

3.18. MI_VIF_IrqType_e

- 说明

定义硬件中断类型

- 定义

```
typedef enum
{
    E_MI_VIF_IRQ_FRAMESTART, //frame start irq
    E_MI_VIF_IRQ_FRAMEEND, //frame end irq
    E_MI_VIF_IRQ_LINEHIT, //frame line hit irq
    E_MI_VIF_IRQ_MAX,
} MI_VIF_IrqType_e;
```

- 成员

成员名称	描述
E_MI_VIF_IRQ_FRAMESTART	VIF Feame start 中断类型
E_MI_VIF_IRQ_FRAMEEND	VIF Frame done 中断类型
E_MI_VIF_IRQ_LINEHIT	VIF LineCount Hit 中断类型

- ※ 注意事项

- E_MI_VIF_IRQ_FRAMESTART: 每一帧第一个 pixel 的信号
- E_MI_VPE_IRQ_ISPFRAMEDONE: 每一帧最后一个 pixel 的信号
- E_MI_VIF_IRQ_LINEHIT: 每一帧指定 line 到达时的信号

- 相关数据类型及接口

[MI_VIF_CallbackParam_t](#)

3.19. MI_VIF_CallbackParam_t

- 说明

定义回调参数

- 定义

```
typedef struct MI_VIF_CallbackParam_s
{
    MI\_VIF\_CallbackMode\_e eCallBackMode;
    MI\_VIF\_IrqType\_e eIrqType;
    MI\_VIF\_CALLBACK\_FUNC pfnCallBackFunc;
```

```
MI_U64 u64Data;
} MI_VIF_CallbackParam_t;
```

➤ 成员

成员名称	描述
eCallbackMode	Callback mode
eIrqType	Hardware interrupt type
pfnCallbackFunc	回调函数
u64Data	回调函数参数

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_VIF_CallbackTask_Register](#)
[MI_VIF_CallbackTask_UnRegister](#)

4. VIF 错误码

视频输入 API 错误码如下表所示。

表 4-3: 视频输入 API 错误码

错误代码	宏定义	描述
0xA0062001	MI_ERR_VIF_INVALID_DEVID	视频输入设备号无效
0xA0062002	MI_ERR_VIF_INVALID_CHNID	视频输入通道号无效
0xA0062003	MI_ERR_VIF_INVALID_PARA	视频输入参数设置无效
0xA0062006	MI_ERR_VIF_INVALID_NULL_PTR	输入参数空指针错误
0xA0062007	MI_ERR_VIF_FAILED_NOTCONFIG	视频设备或通道属性未配置
0xA0062008	MI_ERR_VIF_NOT_SUPPORT	操作不支持
0xA0062009	MI_ERR_VIF_NOT_PERM	操作不允许
0xA006200C	MI_ERR_VIF_NOMEM	分配内存失败
0xA006200E	MI_ERR_VIF_BUF_EMPTY	视频输入缓存为空
0xA006200F	MI_ERR_VIF_BUF_FULL	视频输入缓存为满
0xA0062010	MI_ERR_VIF_SYS_NOTREADY	视频输入系统未初始化
0xA0062012	MI_ERR_VIF_BUSY	视频输入系统忙
0xA0062080	MI_ERR_VIF_INVALID_PORTID	视频输入端口无效
0xA0062081	MI_ERR_VIF_FAILED_DEVNOTENABLE	视频输入设备未启用
0xA0062082	MI_ERR_VIF_FAILED_DEVNOTDISABLE	视频输入设备未禁用
0xA0062083	MI_ERR_VIF_FAILED_PORTNOTENABLE	视频输入通道未启用
0xA0062084	MI_ERR_VIF_FAILED_PORTNOTDISABLE	视频输入通道未禁用
0xA0062085	MI_ERR_VIF_CFG_TIMEOUT	视频配置属性超时
0xA0062086	MI_ERR_VIF_NORM_UNMATCH	视频ADC VIU不匹配
0xA0062087	MI_ERR_VIF_INVALID_WAYID	视频通路号无效
0xA0062088	MI_ERR_VIF_INVALID_PHYCHNID	视频物理通道号无效
0xA0062089	MI_ERR_VIF_FAILED_NOTBIND	视频通道未绑定
0xA006208A	MI_ERR_VIF_FAILED_BINDED	视频通道已绑定

5. PROCFS 介绍

5.1. cat

➤ 调试信息

```
#cat /proc/mi_modules/mi_vif/mi_vif0
```

```
-----dump group attr-----
GroupId  Intf      Work  Clkedge  Hdr
0  BT656 1MULTIPLEX  0  0

-----dump dev attr-----
Dev  IsrCnt  incrop  infmt  AsyncCnt  EnqCnt  BarCnt  CheckCnt  DequCnt
0  3194(0,0,1280, 720)  YUV422UYVY  9556  3188  3188  12741  3186

-----dump pipe attr-----
PipeId  PortId  DevId  Hdn  crop  Dest  Pixel  wkmode  Fbc
1  0  0  0(0,0,1280, 720)(1280x 720)  14  1  0

-----dump outport attr-----
Dev  Port  Cap_size  Dest_size  Fmt  Rate  Atom  MetaInfo  OutCount  Addr0Cnt  FailCount  Fps
0  0(0,0,1280, 720)(1280, 720)YUV422UYVY  0  21000000000000c73  c72  0  0  26

Dev  Port  Recv_size  Out_size  SubOut_size  ReadIdx  WriteIdx  DequeIdx  FrameStartCnt  FrameDoneCnt  VbFail  DropFrameCnt  RingBufStatus
0  0  (1280, 720) ( 319, 719) ( 0, 0)  2  4  2  c74  c72  0  0 (3,3,1,0,3,3,3,3)
```

➤ 调试信息分析

记录当前 VIF 的使用状况以及 Group/ Dev/ pipe/ outport 属性，可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

参数		描述
Group Attr	GroupId	Group 号
	Intf	输入数据的协议
	Work	工作模式
	Clkedge	时钟边沿触发模式，与 Intf 有关系
	Hdr	Hdr 类型
Dev Attr	Dev	Dev 号
	IsrCnt	收到的中断总数
	incrop	Input Crop Size
	infmt	输入数据格式
	AsyncCnt/EnqCnt/ BarCnt/CheckCnt/ DequCnt	Callback 接口执行次数
Pipe Attr	PipeId	Pipe 号
	PortId	Port 号
	DevId	Dev 号
	Hdn	H2T1PMode 使能
	crop	Input Corp Size

参数		描述
	Dest	Output Size
	Pixel	Output Pixel
	wkmode	Pipe 工作模式
	Fbc	Fbc 使能
OutPort Attr	Dev	Dev id
	Port	Port id
	Cap_size	输入 size
	Dest_size	输出 size
	Fmt	Output pixel format
	Rate	Frame rate type
	Atom	底层拿住 buffer 数量
	MetaInfo	Frame id
	OutCount	输出 frame count
	Addr0Cnt	Frame 地址 0 的次数
	FailCount	获取 outputbuffer 失败次数
	Fps	Frame per second
	Recv_size	Vif 硬件收到 size
	Out_size	Write dma size
	SubOut_size	Write sub dma size
	ReadIdx/WriteIdx/ DequeIdx	Read,write,deque 的 index
	FrameStartCnt	处理到第几 frame
	FrameDoneCnt	处理完多少 frame
	VbFail	VbFail 计数
	DropFrameCnt	Drop frame 的数量
	RingBufStatus	Ring buf 状态

5.2. echo

```
# echo help >/proc/mi_modules/mi_vif/mi_vif0
```

```
dump          [devid portid /path];
initatom      [devid InitAtom];
usetimer      [devid bUseTimer];
erasebuffer    [devid portid bEraseBuffer];
devmask       [devid bMask];
resethw       [devid eResetType(0:FIFO 1:wdma 2:AFIFO_WDMA_GROUP 3:FIFO_WDMA_DEV)];
```

Echo help 查看可用命令

功能	Dump frame 到指定路径
命令	echo dump [devid portid /path] > /proc/mi_modules/mi_vif/mi_vif0
参数说明	Devid: dev id
	Portid: port id
	Path: 路径存放
举例	echo dump 0 0 /tmp > /proc/mi_modules/mi_vif/mi_vif0

功能	设置 vif 的 atom 数量
命令	echo initatom [devid InitAtom] > /proc/mi_modules/mi_vif/mi_vif0
参数说明	Devid: dev id
	InitAtom: Driver 最大持有 buffer 数量
举例	echo initatom 0 2 > /proc/mi_modules/mi_vif/mi_vif0

功能	设置 timer
命令	echo usetimer [devid bUseTimer] > /proc/mi_modules/mi_vif/mi_vif0
参数说明	devid: device id
	bUseTimer: 使能 timer
举例	echo usetimer 0 1 > /proc/mi_modules/mi_vif/mi_vif0

功能	擦除 vif 缓存
命令	echo erasebuffer [devid portid bEraseBuffer] > /proc/mi_modules/mi_vif/mi_vif0
参数说明	devid: device id
	portid: port id
	EraseBuffer: 使能擦除缓存
举例	echo erasebuffer 0 0 1 > /proc/mi_modules/mi_vif/mi_vif0

功能	设置 Dev Mask
----	-------------

命令	echo devmask [devid bMask] > /proc/mi_modules/mi_vif/mi_vif0
参数说明	devid: device id
	bmask: 使能 mask
举例	echo devmask 0 1 > /proc/mi_modules/mi_vif/mi_vif0